AI IN SDLC WORKSHOP

What this assignment is

You will produce the four things a senior engineer should own before a service goes to production: a decision record, an autonomy policy, a working evaluation, and an honest measurement plan. Everyone works in the same scenario and the same codebase, so the results can be compared and argued about rather than taken on trust.

Nothing here has a single right answer. Two people who choose differently from the same question can both do well. What separates a strong submission from a weak one is whether the choice is defended against its strongest objection, and whether the numbers behind it were measured rather than assumed.

Use whichever approved agent and tools you have.

The scenario

The codebase is ai-in-sdlc-assignment . The scenario around it is in `ops/README.md` in that repository, along with the evidence packs you will need. In short:

You have just been assigned as the senior engineer for the Invoice Approval service. It has run for 18 months as an internal tool maintained part-time by one person. In January it becomes a tier-2 service: a 24x7 on-call rotation, a published SLO, and a new integration that submits approved invoices to an external payment provider, PayEx.

Fact	Detail
Team	Six engineers, two new to this codebase, one-week on-call rotation
Platform	ASP.NET Core 8 on Azure App Service, Azure SQL, GitHub Actions
Available but unused	Azure Service Bus is provisioned, and nothing uses it yet
Volume	~40,000 invoices a month, 60% in the last three working days, peaking near 80 approvals a minute
Persistence today	None. The repository uses an in-memory store, so the database is part of going to production
PayEx behavior	Returns a reference on success. Times out at 30 seconds, and a timeout does not reliably mean the submission failed
Compliance	SOC 2 Type II audit in six months. Approval actions auditable for seven years

Everything in `ops/` describes this planned deployment, not the code as it stands. Keep that distinction: it is the subject of the first task.

What to do

1. **An as-built diagram with citations, and one architecture test.** Use the Mermaid MCP to produce a container view and a sequence diagram for the approval path, citing a file or symbol for every element, and list separately everything the agent inferred. Correct whatever is wrong. Then draw the target state once persistence and PayEx are in, marking clearly which parts do not exist yet. Finally, encode one structural rule as a test that actually runs, with the command written down. One test that runs beats five that are described.

2. **A decision record on exact-once submission.** How do you guarantee an approved invoice reaches PayEx exactly once? Consider at least an idempotency key with a dedupe table, a transactional outbox with a worker, and optimistic submission with nightly reconciliation. Have the agent lay out the trade-offs, then stress it three ways: state a preference for a different option and see whether the recommendation moves; ask how your choice fails at 3 a.m. on the last day of the month; ask what it costs to build and to run. Write the record yourself, including what would make you revisit it, and a section headed *Where the agent was wrong or useless*, with specifics.

3. **An autonomy policy for one workflow.** Pick one: month-end log triage, dependency bumps, or first-response incident summaries. Write one page covering which tasks sit at which level of autonomy, the controls that make that level safe, the evidence needed to move a task up a level, and who approves the move. Then state which level this team can run on day one of the rotations, given that two of the six engineers are new to the codebase.

4. **A review evaluation, run three times.** The repository has ten fixed cases in ops/evals. Run the whole set three times against one review to prompt and report caught, missed and false alarm rates rather than anecdotes. Four of those cases were shown in the session, so add two cases of your own from real defects you have seen, at least one of them clean. Then change exactly one thing — the prompt, the model, or the context file — and run the set again. Say whether the change was an improvement, and how confident you are at that sample size.

5. **A risk audit and a measurement plan.** Audit the agent setup described at the end of ops/README.md against the lethal trifecta: name which legs are present, then either redesign it to remove one or accept the risk in writing with the trigger that would make you revisit. Separately, write the baseline you would capture before the rotation starts — lead time for changes, change failure rate, review turnaround — naming the exact source or query for each, and actually capture the two you can get today from the repository and its CI history.

The demo video

Record one demo video of 8 to 10 minutes using Teams or using any screen recorder that produces an MP4.

What to hand in

1. Mermaid source for the as-built and target diagrams, with the citation list and the inferred-items list
2. One architecture test, with the command that runs it
3. The exactly-once decision record, including the Where the agent was wrong or useless section
4. A one-page autonomy policy, with the day-one level stated

5. Your two added evaluation cases, and a results table covering three runs plus the changed-variable run
6. The risk audit, with a decision recorded
7. The measurement plan, with the two numbers you captured
8. The 8-to-10-minute demo video, uploaded and linked
9. AI Augmentation logs should be provided